

Section 7.1: Enterprise Integration Styles

Study Set: 3 files total | **Section:** 7.*

Focus: Integration patterns, API protocols, and architectural evolution for system design interviews.

Executive Overview

Integration patterns define how distributed components communicate. Mastery of these patterns — from legacy ESBs to modern event-driven architectures — is foundational for system design interviews at senior and staff levels. This section covers when and why to choose each pattern, with concrete trade-offs and interview-ready answers.

1. Architectural Communication Evolution

1.1 Legacy N-Tier (Monolithic) Architecture

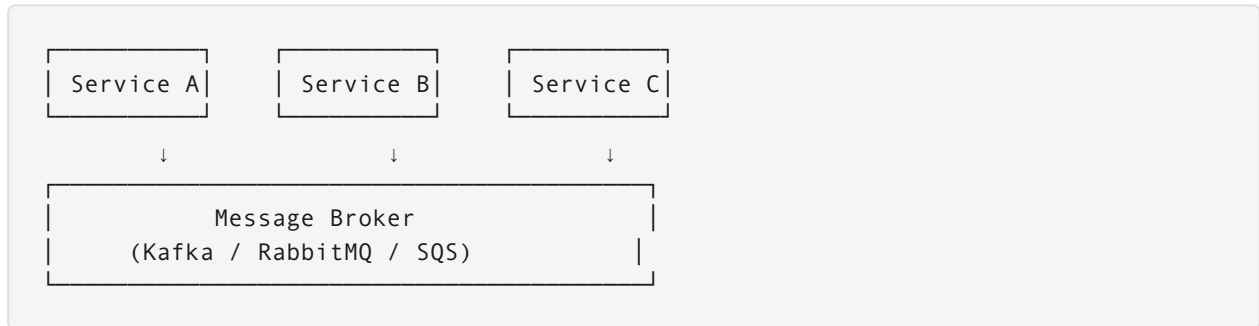
```
Client → Web Server → Business Logic → Database
All components in same process
Direct method calls / Shared memory for state
```

Why it fails at scale:

Problem	Impact
Tight coupling	One change requires full redeploy
Vertical-only scaling	Cannot scale hot components independently
Single point of failure	Any layer down = total outage
Technology lock-in	Stack upgrade = full system risk

Interview signal: If asked to “improve a monolith,” identify which domain boundaries can be extracted first (e.g., auth, notifications) — this becomes the microservices decomposition strategy.

1.2 Microservices Architecture



Benefits vs. Challenges:

Benefit	Challenge
Independent scaling per service	Distributed tracing complexity
Technology diversity per team	Network latency between hops
Fault isolation	Data consistency across services
Team autonomy	Operational overhead (k8s, service mesh)

When to prefer microservices: Teams >10 engineers, domains with very different scaling profiles (e.g., read-heavy catalog vs write-heavy checkout), regulatory isolation requirements.

When to stay monolithic: Early-stage products, unclear domain boundaries, small teams — a “modular monolith” is often the right intermediate step.

2. API Protocol Paradigms — Comparison & Decision Guide

This is one of the most common system design interview topics. The key is knowing *when* to use each, not just *what* each is.

2.1 REST/HTTP

Mechanics: - Stateless request/response model - Standard HTTP verbs: GET , POST , PUT , DELETE , PATCH - JSON/XML payloads (human-readable) - Cacheable via Cache-Control , ETag , Last-Modified headers

Performance overhead per request:

```
TCP handshake:      ~1 RTT (waived with keep-alive)
TLS negotiation:    ~1-2 RTT (waived with session resumption)
HTTP headers:      ~1KB per request
JSON parse (1MB):  ~100ms CPU
```

Use REST when: - Exposing a public API (third-party developers expect it) - Browser-first clients (native support, no proxy) - Team tooling matters (curl, Postman, browser DevTools) - Response caching is valuable

2.2 gRPC (Protocol Buffers over HTTP/2)

Schema-first contract:

```
message GetUserRequest { int64 user_id = 1; }

message User {
  int64 id      = 1;
  string name   = 2;
  string email  = 3;
}

service UserService {
  rpc GetUser(GetUserRequest) returns (User) {}
  rpc WatchUsers(google.protobuf.Empty) returns (stream UserUpdate) {}
}
```

Performance comparison:

```
REST JSON  - 1MB payload → ~1MB wire, ~100ms parse
gRPC proto - 1MB payload → ~200KB wire, ~10ms parse
Net gain:   5× CPU savings, 5× bandwidth reduction
```

Use gRPC when: - High-throughput internal service-to-service calls - Native streaming needed (server push, bidirectional) - Strong interface contract enforcement at compile time - Service mesh architecture (Envoy understands gRPC natively)

Limitations: - Binary protocol — not human-debuggable without tooling - Browsers require gRPC-web proxy layer - HTTP/2 dependency adds ops complexity

2.3 GraphQL

Schema-driven, client-controlled queries:

```

type Query {
  user(id: ID!): User
}
type User {
  id: ID!
  name: String!
  posts: [Post!]!
}

# Client fetches only what it needs:
{ user(id: "123") { name posts { title } } }

```

Use GraphQL when: - Multiple client types (mobile, web, partners) with different data needs - Avoiding over-fetching / under-fetching across client types - Rapid frontend iteration without backend API changes

Known pitfalls: - **N+1 query problem:** fetching a list of users + their posts fires N+1 DB queries — mitigate with DataLoader (batching) - **Caching complexity:** single endpoint breaks HTTP cache semantics; use persisted queries + CDN key on query hash - **Authorization complexity:** field-level auth must be explicit per resolver

2.4 Protocol Decision Matrix

Factor	REST	gRPC	GraphQL
Development speed	Fast	Slower (IDL + codegen)	Medium
Debugging	Easy (curl, browser)	Hard (binary)	Medium (GraphiQL)
Performance	Moderate	High	Moderate
Browser support	Native	Needs proxy	Native
Streaming	Polling/WebSocket workaround	Native (4 modes)	Subscriptions
Schema evolution	Flexible	Strict (versioned proto)	Additive-only
Best for	Public APIs	Internal services	Multi-client APIs

Hybrid architecture (recommended for most large systems):

```

External clients
  ↓
API Gateway (REST/GraphQL — developer-friendly)
  ↓

```

```
gRPC adapter
  ↓
Internal services (gRPC – performance-optimized)
```

This gives you external simplicity and internal performance without forcing service owners to expose REST.

3. Interview Design Problems

Problem 1 — Choose a Protocol for a Fintech Microservice Platform

Scenario: You're designing a payment processing platform with: (a) a mobile app, (b) third-party merchant integrations, and (c) 20 internal microservices. Traffic is 50K RPS internal, 5K RPS external.

Solution approach:

Layer 1 — External API (REST): - Merchant and mobile SDKs expect REST - Standard HTTP auth (OAuth2 + API keys) - OpenAPI documentation is table-stakes for third parties

Layer 2 — Internal service mesh (gRPC): - Payment, fraud, ledger, notification services communicate via gRPC - Protocol Buffers enforce schema contracts — breaking changes caught at compile time - HTTP/2 multiplexing reduces connection overhead at 50K RPS

Layer 3 — Real-time feeds (gRPC streaming):

```
rpc StreamTransactionEvents(Empty) returns (stream TransactionEvent) {}
```

- Fraud detection subscribes to live transaction stream
- No polling overhead

Result: REST externally for ecosystem compatibility, gRPC internally for performance and correctness.

Problem 2 — Migrating a Monolith to Microservices Without Downtime

Scenario: Legacy e-commerce monolith, 3M users, zero downtime requirement. How do you decompose?

Solution approach — Strangler Fig Pattern:

Phase 1 – Route by feature via gateway:
All traffic → Gateway
 ├ /checkout/* → NEW checkout microservice
 └ /* → legacy monolith (unchanged)

Phase 2 – Sync data while dual-writing:
Monolith writes → DB (legacy)
 → Event bus → new service DB (shadow)

Phase 3 – Validate parity, then cut over routing

Key risks and mitigations: - **Data consistency during dual-write:** Use CDC (Change Data Capture via Debezium) rather than app-level dual-write — source of truth stays the monolith DB until cutover - **Rollback plan:** Gateway routing rules can redirect 100% traffic back to monolith in <1 minute - **Interface contract:** Define REST/gRPC contracts *before* migration, so monolith and new service satisfy the same spec

Problem 3 — GraphQL N+1 at Scale

Scenario: Your GraphQL API serves `users` with `posts`. Load testing shows 10K DB queries/second for a feed of 100 users. Fix it.

Root cause:

```
query { users { posts { title } } }  
→ 1 query for users  
→ 100 queries for posts (one per user)  
= 101 queries, scales as O(N)
```

Solution — DataLoader batching:

```
const postLoader = new DataLoader(async (userIds) => {  
  // ONE query for all users' posts  
  const posts = await db.query(  
    'SELECT * FROM posts WHERE user_id = ANY($1)', [userIds]  
  );  
  // Group by user_id and return in same order as userIds  
  return userIds.map(id => posts.filter(p => p.userId === id));  
});  
  
// Resolver now uses loader  
User: {  
  posts: (user) => postLoader.load(user.id) // batched automatically  
}
```

Result: 101 queries → 2 queries, regardless of list size. This is the canonical GraphQL scaling answer interviewers look for.

4. Key Concepts Cheat Sheet

Concept	One-line definition
Stateless REST	Server holds no session; client sends full context each request
HTTP/2 multiplexing	Multiple streams over single TCP connection (gRPC requires this)
Proto versioning	Adding fields is backward-compatible; removing or renumbering is breaking
N+1 problem	O(N) DB queries from a single GraphQL list resolver; fix with DataLoader
Strangler Fig	Incrementally replace a monolith by routing traffic to new services
Service mesh	Infrastructure layer (Envoy sidecars) handling retries, mTLS, tracing
API gateway	Entry point for auth, rate limiting, routing, protocol translation

Next: Section 7.2 — Asynchronous Messaging & Event-Driven Topologies